

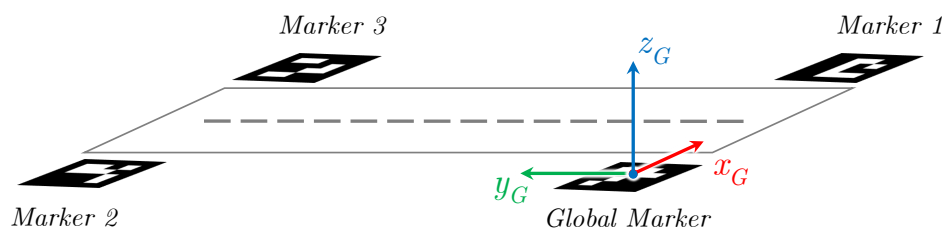
Task 2 – Detecting the Markers Pose

Task 2 is a continuation of the first coding block that focuses on detecting the ArUco-markers. In this task, we will determine the coordinates of the ArUco-markers contained in the video feed relative to the camera, and then, in a second step, transform those coordinates so they are relative to the so called "Global" ArUco-marker (highlighted in the figure below).

Overall, Task 2 will involve:

1. Detecting the pose of ArUco-markers relative to the camera
2. Transforming the detected coordinates to be relative to the global marker
3. Implementing a 3D plotter (optional but helpful)

To better understand why we process the coordinates this way, we need to introduce our actual use case for ArUco-Markers. In our scenario, the ArUco-markers represent the corners of a runway and are therefore arranged in a rectangular pattern. However, since we want our coordinate system to be based on the runway (Global) rather than the plane, or technically, camera (Relative), we must rotate and transform the coordinate system from the camera's perspective to the runway (Global) reference frame. The details of this transformation is described in following graphics:



For this Task we ask you to create a function called `pose_detect` that analyses each detected marker's pose and returns their global coordinates. To do this, inside the `pose_detect` function you will have to implement another function called `pose_transform`. This function will take the relative coordinates and transform them to be global. We believe that it is helpful for this task to create a 3D-Plotter. This plotter shall take four coordinates and have the ability to plot them in a coordinate system. This will be of great help for you, when running into issues with your coordinate transformation. For this Task the solution shall require:

- A 3D-Plotting function (optional, but may be rewarded with bonus points)
- A clear structure inside `pose_detect`
- A function called `pose_transform` that transforms relative to global coordinates
- The completed implementation into previously created `main.cpp`
- A detailed Wiki describing your process

```
(*relative_coordinates[coords_glob_cam, coords_glob_m0, coords_glob_m1,
↪ coords_glob_m2, coords_glob_m3]) = pose_detect(frame, *corners, *ids,
↪ camera_matrix, dist_coeffs);

(*global_coordinates[coords_glob_cam, coords_glob_m0, coords_glob_m1,
↪ coords_glob_m2, coords_glob_m3]) =
↪ pose_transform(*relative_coordinates[coords_rel_cam, coords_rel_m0,
↪ coords_rel_m1, coords_rel_m2, coords_rel_m3]);
```